MANCHESTER 1824
The University of Manchester

Department of Electrical and Electronic Engineering

Embedded Systems Project 2024-25

FINAL REPORT

Buggy Development and Performance Evaluation

Group Number: 39

Group members:	ID Number	I confirm that this is the group's own work.
Emmanuel Pacholek	11439049	☒
Jamie Ashton	11071316	☒
Junbyung Park	10936972	☒
Yik Hay Ryan Ho	11416243	☒
Zhe Shang	11056225	☒

Tutor: Andrew Forsyth
Date: 09/05/2025

Contents

1. Executive Summary 1

2. Introduction 1

3. Final System Components Summary 3

4. Team Organisation and Planning 9

5. Budget vs. Outturn & Business Case 12

6. Analysis of Heats 13

7. References 15

8. Appendix 16

1. Executive Summary

This report outlines the development of a line-following autonomous buggy based on the STM32F401RE micro-controller, completed as part of the Embedded Systems group project. The development process followed a structured sequence of stages, covering hardware and software design, component selection, prototyping, integration, and performance testing.

During the early design phases DR1 and DR2 focused on evaluating motors, sensors, and gearbox options to determine the optimal configuration. These were followed by Technical Demonstrations (TD1 to TD4), including the final heats, to verify full system functionality and track readiness. In parallel, a project proposal explored a potential industrial application of the buggy. A Gantt chart was used to plan and manage project timelines, serving as a reference point for task allocation and team progress.

The report covers key technical aspects of the project: mechanical assembly, sensor and PCB integration, PID-based control logic, state machine implementation. Additional sections include budget and Business case analysis, team collaboration, and reflections on Heats performance.

The project achieved full mechanical and software integration into a compact and functional buggy platform, with successful line-following performance demonstrated during testing. However, the final heats revealed critical challenges. Reduction from 6 sensors to 4 before the event, significantly reducing detection coverage. Limited time and access during the Easter break prevented proper repairs, and the team recalibrated the PID controller using the remaining four sensors. Despite best efforts, the buggy failed to complete the track across all three attempts. Stopping prematurely in the first, losing the line in the second, and veering off due to excess speed in the third.

These outcomes highlighted important lessons in hardware robustness, risk mitigation, and planning under constraints. The absence of redundancy in critical components and reliance on a single member for late-stage tuning introduced bottlenecks that compromised reliability. In retrospect, a more test-driven and modular development approach, combined with spare components and distributed access, would have supported greater resilience and broader team involvement.

The report concludes with individual Continuing Professional Development (CPD) logs. Overall, the project provided valuable hands-on experience in embedded system design, sensor and motor testing, PCB layout, control algorithm tuning, hardware debugging, and team-based engineering—skills directly applicable to future work in robotics, automation, and embedded systems.

2. Introduction

This report forms part of the Embedded Systems Project and aims to provide a comprehensive overview of the final design of the project buggy in terms of hardware, software, financial considerations, and performance in technical demonstrations. It serves as a summary of key findings, as well as a reflection on the design process and collaboration—offering insights to inform future projects of a similar nature.

The Embedded Systems Project offers valuable hands-on experience in applying practical engineering principles while also incorporating elements of business planning. It challenges participants to lead and collaborate within a team environment, navigating the full design cycle—from initial brainstorming and conceptual development to prototyping, validation, and final assembly. The project also encourages flexible and adaptive strategies to overcome unforeseen technical and logistical challenges.

Throughout the project, several deliverables were produced, including Design Reports 1 and 2 (DR1–2), multiple Technical Demonstrations (TD1–4), a project presentation, and this final report. The key deliverable of the project is the buggy as shown in Figure 1.

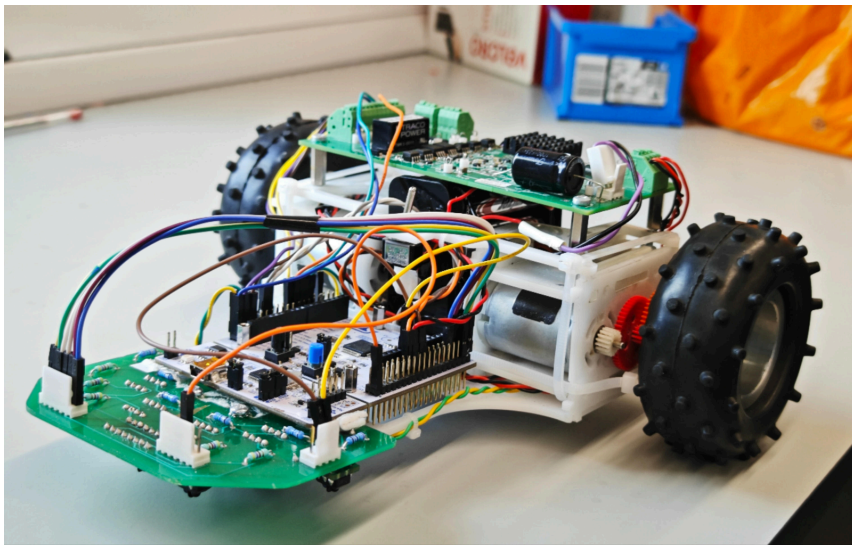

Figure 1: A picture for the final buggy

This report is structured into four main sections:

1. **Final System Components Summary** – This section outlines the final hardware and software architecture of the buggy, detailing the structural, electrical, and embedded system designs. It also highlights key iterative changes made during the development process, showcasing decision-making, problem-solving, and the evolution of the design.
2. **Team Organisation and Planning** – This part focuses on the collaborative and managerial aspects of the project, describing how the team structure and objectives evolved in response to a dynamic project environment. It reflects on planning strategies, adaptability, and lessons learned.
3. **Budget vs. Outrun** – This section examines the financial aspects of the project, including actual expenditures for design, prototyping, and assembly. It also considers potential market pricing if the buggy were to be commercialised.
4. **Analysis of Heats** – The final section reviews the buggy's performance in the technical demonstrations. It includes preparation strategies, performance outcomes, and critical reflections on what worked well and what could be improved upon in future iterations.

3. Final System Components Summary

3.1. Software Design

Class Structure

Distinct classes were implemented for motor control, the encoders, the Bluetooth functionality, the sensor array, the controller, and the buggy. Abstraction was used well to only show high-level functionality which made implementing these classes very simple. Each class has a focused purpose, and this made the code much easier to tune and test.

State Machine

The buggy's behaviour was managed using a state machine with defined states: line-following, line-lost, stopping, and turnaround. Each state had dedicated functions and clear entry/exit actions, with flags enabling reliable transitions. This structure ensured predictable and maintainable control logic.

Line Detection

The code features line detection through 6 analogue inputs from the sensors, background subtraction is carried out (the value of the sensor reading with the IR LED on minus the value of the sensor reading with the IR LED off) to prevent background illumination affecting the readings. Each sensor was assigned an x-offset to represent its position, and sensor readings were weighted accordingly. A weighted average was then computed to estimate the line's position. To detect line presence, a minimum weight sum threshold was implemented, filtering out noise when no line was detected.

Line Breaks

The code can handle small gaps in the line by maintaining its previous direction until the line is found again. This is dictated by a 'line-lost timer', which is kept low enough so that small gaps are crossed, but the buggy comes to a complete stop at the end of the track.

Bluetooth Functionality

The code made use of the buggy's HM-10 BLE module by implementing a Bluetooth class, turnaround flags and the encoders to allow a Bluetooth symbol to trigger an exact 180 degree turn-around.

3.2. Buggy Mechanical Design

The final mechanical design focused on creating a compact layout with a low centre of gravity to improve stability and manoeuvrability. A redesigned chassis was introduced, allowing the heavy components such as the battery pack to be positioned between the gearboxes. This change resulted in a more balanced weight distribution, with the centre of mass placed lower and closer to the drive shaft. As a result, the buggy achieved faster directional changes and smoother transitions during motion.

The previous chassis did not present any mechanical faults, but the updated design offered a more efficient use of space and improved performance. Its flatter profile allowed the sensor array to remain closer to the ground, reducing vertical vibration and improving measurement consistency. This contributed to more reliable sensor readings and quicker response from the PID control system, particularly during cornering, where stability and responsiveness are critical.

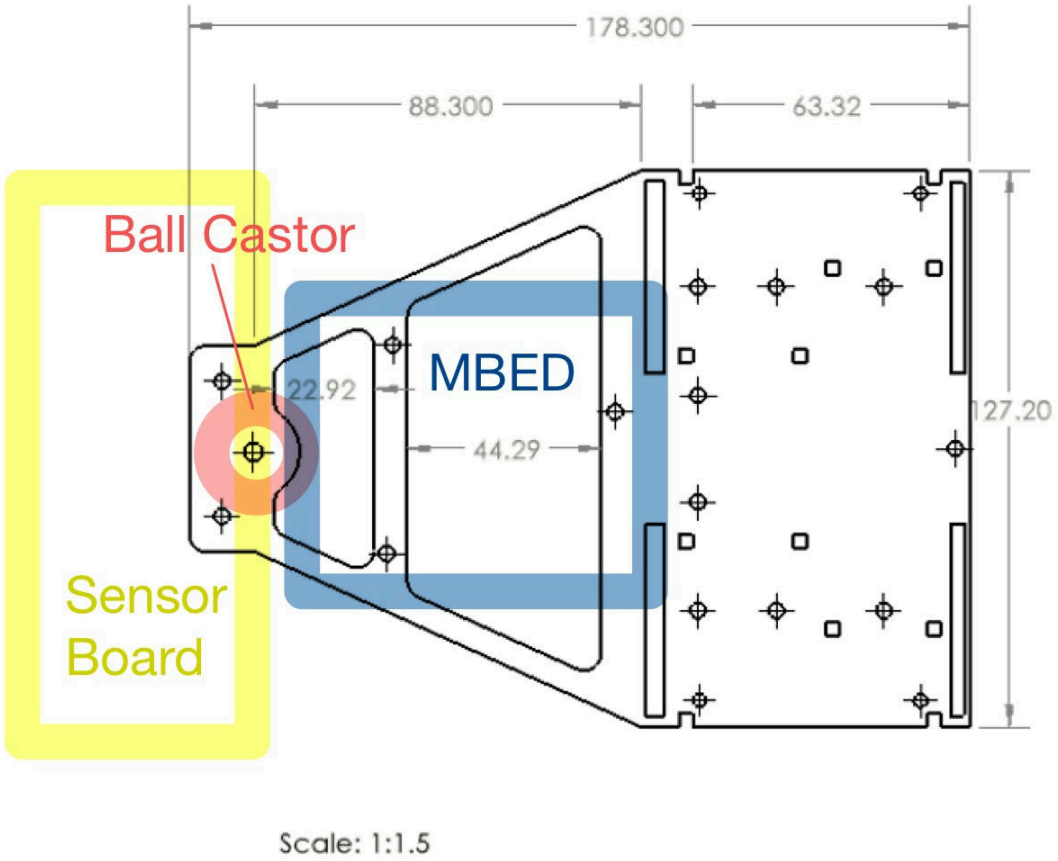

Figure 2: Design of the previous chassis

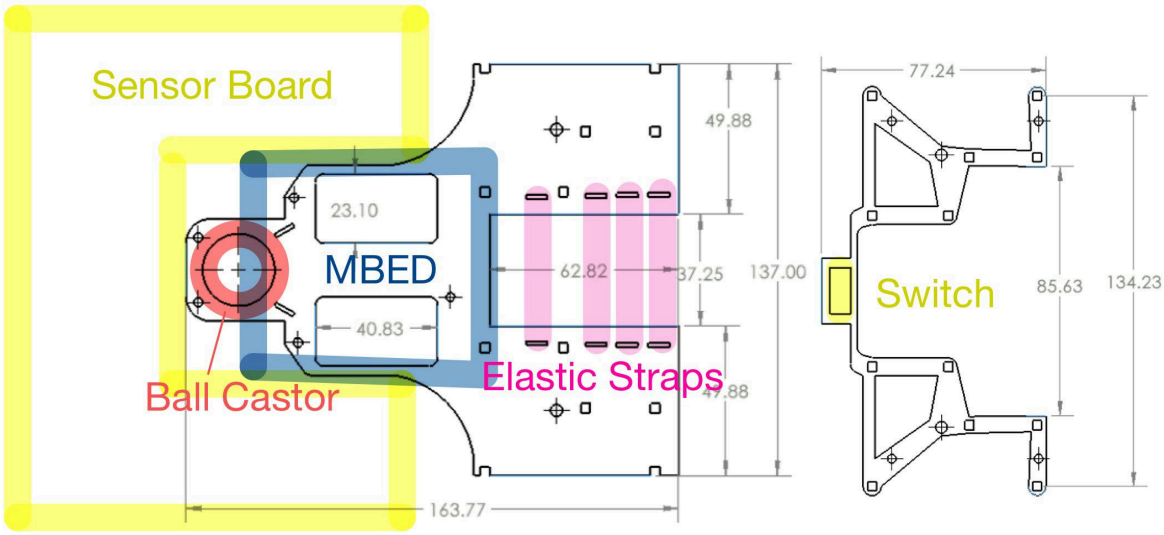

Figure 3: Design of the new chassis

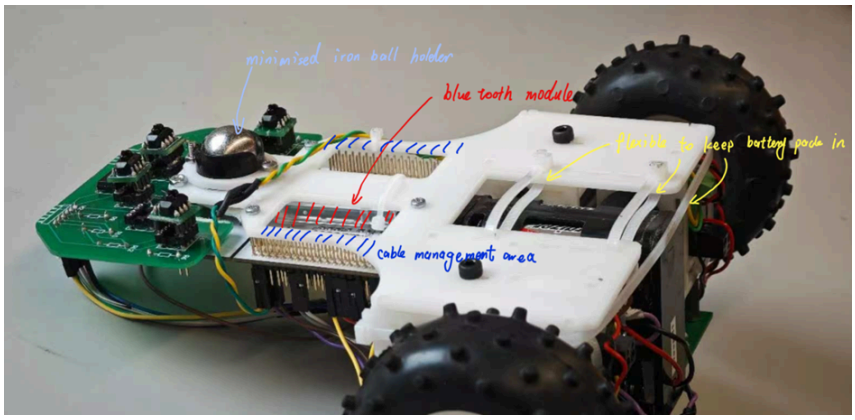

Figure 4: The bottom view of the buggy

To shorten the overall vehicle length, we employed standoffs of varying lengths, enabling the front sensor board, the mbed controller board, the chassis base, and the exterior support frame to stack compactly while preserving the sensor board's adjustable mounting height.

Mass reduction of the chassis was achieved by replacing long, bulky screws originally securing the support frame's steel ball with by a "sandwich" assembly in which the frame is bolted directly through clearance holes in the chassis, saving roughly 80 g. Directly beneath the mbed board, additional cutouts were introduced to remove excess material, yet the mounting zone for the Bluetooth module was retained and secured by a slender cable tie. Although not employed in the final prototype, two lateral chassis rails flanking the mbed's pin headers were incorporated to facilitate wire management during debugging. At the chassis rear, coarse elastic straps replaced rigid sections beneath the battery pack—this not only reduced weight but also provided a secure yet easily removable battery mount. The protective plastic cover above the motor-controller board was removed both to save mass and to serve as the battery compartment's upper enclosure.

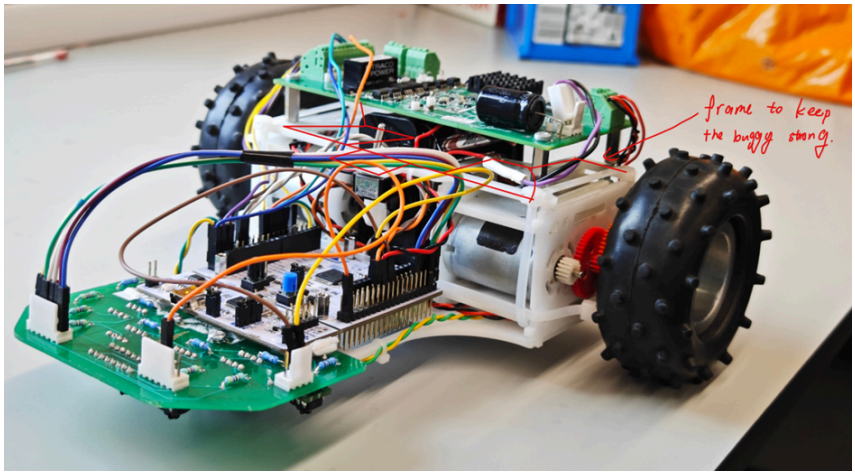

Figure 5: 45 degree down view of the buggy

To further enhance structural integrity without adding substantial weight, an upper support frame was designed to link protrusions on each motor, an arrangement taking inspiration from performance automobiles' strut-tower brace between the front suspension towers like the BMW M3. Together with the lower chassis (functionally akin to a subframe cross-member), this overhead brace forms a robust, torsion-

resistant chassis assembly. Moreover, the upper cross-member doubles as the mounting base for both the motor-controller board and the main power switch, consolidating multiple functions into a unified, high-rigidity structural element.

3.3. Sensor Board Design

Before TD2, the group's aim was to have a working PCB for the sensor array, rather than a strip-board circuit. Due to lack of time the first board was a simple straight array of 6 TCRT5000 sensors, whose LEDs were driven by the ULN2003 Darlington Array IC.

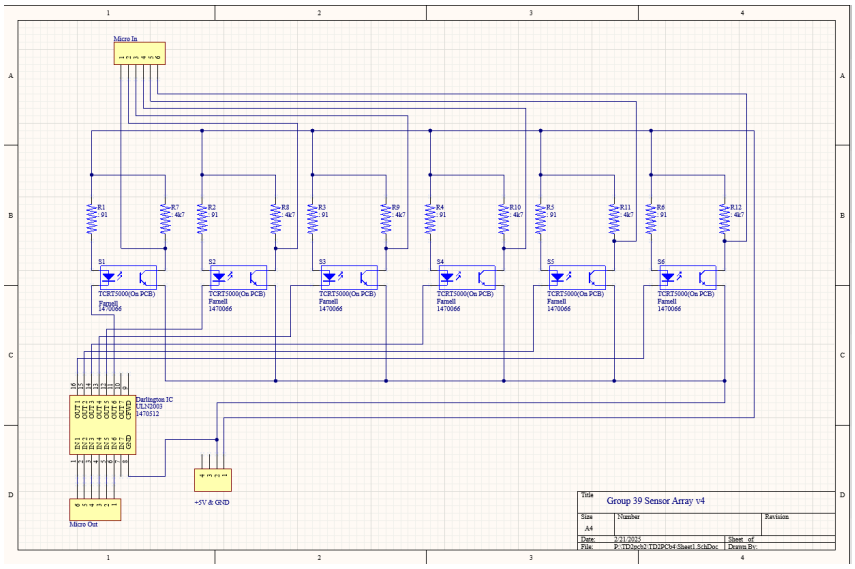

Figure 6: First schematic for the sensor array

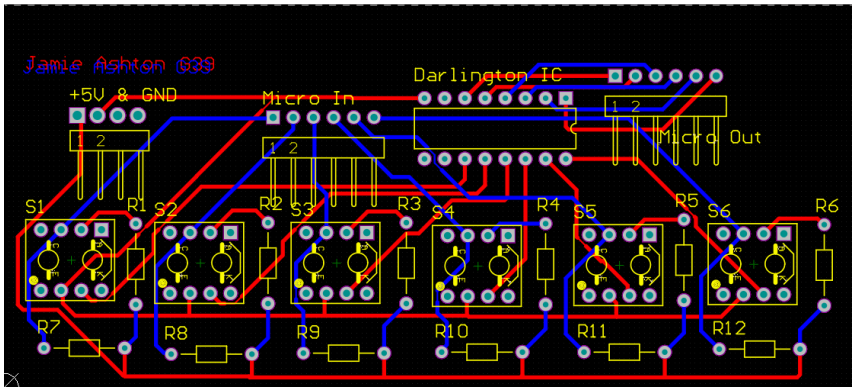

Figure 7: First PCB design for the sensor array

R1 – R6 were the current limiting resistors, and their values were chosen to give a forward current of around 40 mA. R7-R12 were chosen to convert the sensor output to a useable analogue voltage. The distance between the sensors was chosen based on experimental results from semester 1 and ensured that at least one sensor would always see the line, even at extreme angles. Once this PCB was printed it was assembled, wired with the buggy, and tested with our TD2 code. It performed well however there were a few adjustments that needed to be made.

For this design the phototransistors were in a common-collector configuration, this meant that when over the white line they returned low values, and when over the black track returned high values, this would be counterintuitive for our software

implementation. Secondly the LED forward currents were found to be 20 mA instead of 40 mA due to voltage drops across the ULN2003. Lastly, we felt that we wanted to implement an arc shape for the board that would fit around the ball castor allowing the sensors to detect curves in the line more promptly.

In-between TD2 and TD3 we prototyped and tested a new sensor board. This board used the common-emitter configuration, used a lower value of current-limiting resistor 75 Ω and an output resistor value of 4k7 Ω chosen so that the sensor readings used nearly the full range of the micro-controller's ADC (3.3 V). The board also had a refined shape that fit well with the new chassis design. This board was constructed and we verified its operation.

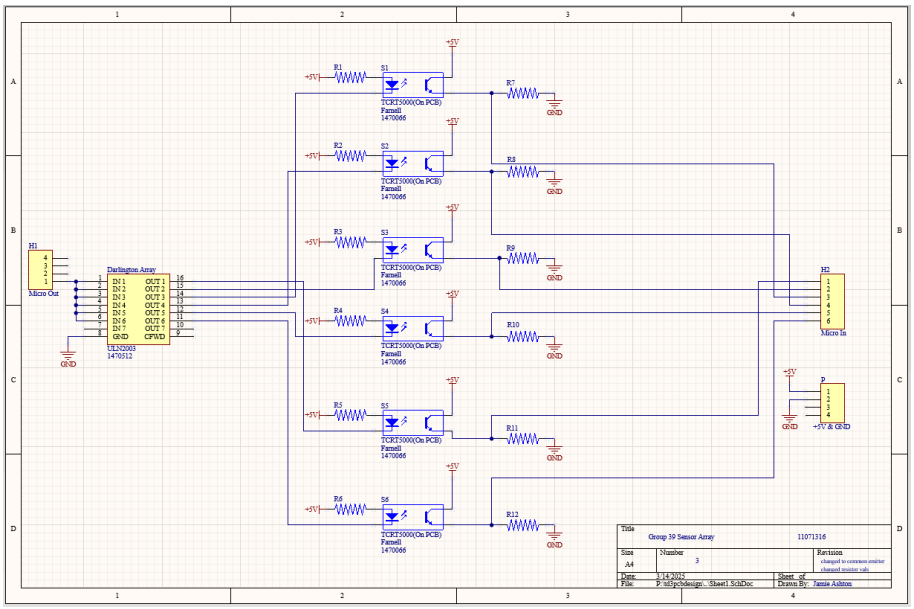

Figure 8: Second schematic for the sensor array

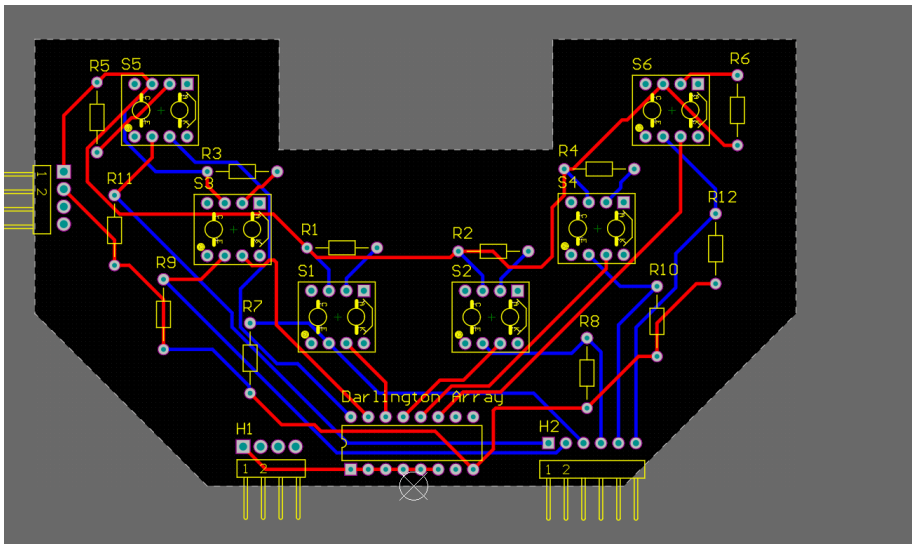

Figure 9: Second PCB design for the sensor array

3.4. Wiring of the Buggy

The buggy's wiring was designed for minimal length and compact layout to reduce resistance losses and maintain a tidy structure. The motor driver board and STM32 micro-controller were oriented to allow the shortest possible wiring paths while accommodating the placement of the sensor board and other peripheral components. This arrangement matched the orientation shown in the wiring diagram.

The wiring design underwent multiple iterations throughout the project. Initially, the application shield was used to demonstrate sensor interactivity and validate readings. However, due to the limited number of available analog input pins on the micro-controller (required for continuous sensor data acquisition) the application shield was removed. This decision also supported efforts to reduce power loss and optimise performance. The removal of the shield in TD3 prompted a complete redesign of the wiring system, with the resulting layout shown in Figure 10.

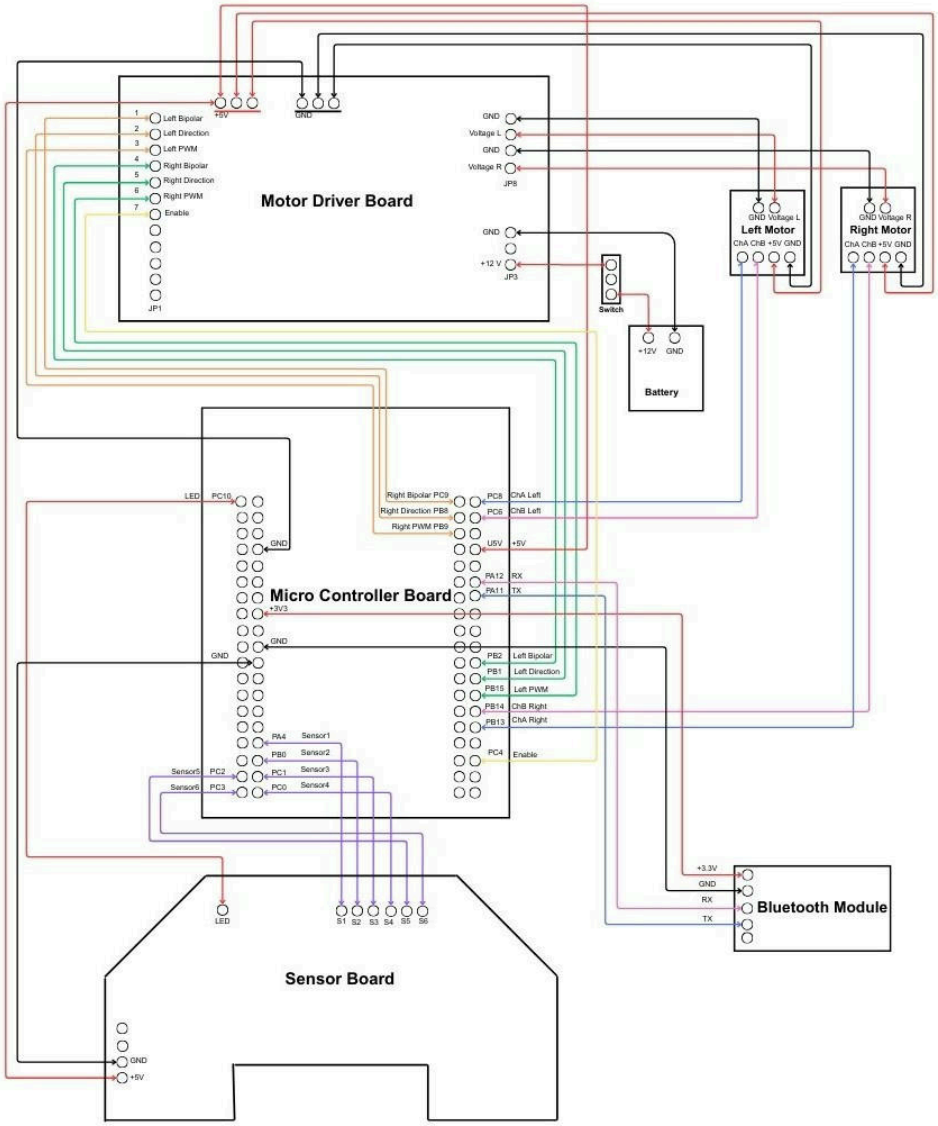

Figure 10: Wiring Diagram of the Final Buggy

4. Team Organisation and Planning

4.1. Project Objectives

At the outset of the project, the team shared a clear and ambitious objective: to achieve first place in the final race. The project goal shaped the overall planning strategy, driving a dual focus on both speed and reliability for the line-following buggy. The development process was structured around producing high-quality design documentation(DR1 and DR2) and meeting the technical milestones defined by each demonstration phase, culminating in the final qualification stage.

Despite the initial ambitions, the project encountered several unexpected setbacks that required a significant shift in objectives and strategy. Damage to the sensor array (specifically, broken sensors obstructing the PCB socket) compromised the reliability of the control system. As a result, the team's focus moved from aiming for top race performance to ensuring a consistent and dependable run on the heats track.

To mitigate risk and prioritise reliability, the tuning strategy is adjusted to operate at lower speeds. This approach helped establish a stable baseline performance under the constraints faced. Additional modifications and refinements were made dynamically throughout the Heats phase, as outlined in the Analysis of Heats section. These adaptations highlight the team's flexibility, resilience, and ability to respond effectively to unforeseen challenges under pressure - a good approach for future projects when facing setbacks.

4.2. Responsibilities

Responsibilities are divided based on individual strengths and interests. For the early design stages, this worked effectively: each member contributed to different aspects, including chassis design, sensor selection, software architecture, and documentation. This allowed the team to meet the deadlines for Design Reports 1 and 2 with satisfactory results, fostering good collaboration and shared ownership of the project.

However, as the project progressed from design to implementation—particularly during the Technical Demonstrations—the workload became significantly more technically demanding, especially in software development and system tuning. At this stage, our team dynamics began to shift. Given the relatively small but complex code-base, which required continuous tuning and iterative problem-solving, it became increasingly difficult to divide the programming tasks evenly. As a result, two members with stronger coding backgrounds naturally took on a larger share of the implementation and debugging work, while others contributed more intermittently or shifted toward supportive roles.

This imbalance could have been mitigated with more structured software planning, clearer task breakdowns, and better documentation from the outset. Incorporating a real-time collaborative platform and more deliberate task management strategies would also have helped distribute the workload more evenly and supported wider team involvement during critical implementation phases.

4.3. Setbacks

Unforeseen technical challenges significantly disrupted our initial project schedule. Although the Gantt chart outlined structured deadlines aligned with each deliverable,

the practical integration of hardware and software systems introduced complexities that led to substantial redesigns under time pressure.

Redesign of the Buggy

Following Technical Demonstration 2, it became clear that early design decisions, particularly in sensor integration and motor control, were not sufficient to meet the performance targets. This realisation led to a full-scale redesign of the chassis, sensor board, and control logic ahead of Technical Demonstration 3. The change compressed multiple stages of development into a short timeframe, requiring continuous iteration with limited room for error.

A key issue emerged during this phase: constant refinement, while well-intentioned, caused project churn. Most of the systems developed for TD2 were discarded, and with core components being rebuilt just days before a major milestone, the workload intensified, including multiple overnight sessions to complete integration.

Hardware Failure

The situation worsened with a hardware failure, specifically, a broken sensor, shortly before TD3. The fault critically affected the control system due to the lack of sensor redundancy, leaving limited options for recovery within the available timeframe. This disruption significantly affected both testing and final preparation.

Additional constraints arose during the Easter break. With only one team member able to take the buggy offsite, ongoing development and tuning became restricted to their availability, reducing overall team productivity. These limitations prompted a strategic shift: instead of optimising purely for speed, the focus was redirected toward reliability and meeting qualification criteria for TD4. To support this change, the software was restructured and simplified, prioritising consistent performance over experimental features.

4.4. Reflections

Team Planning

While the Gantt chart provided clarity for initial deliverables, its rigidity hindered adaptability during rapid redesign phases (e.g., recalibrating PID parameters post-hardware changes). Transitioning to an iterative, test-driven workflow improved technical flexibility, but the lack of structured role definitions led to inefficiencies in task ownership and accountability.

In retrospect, a hybrid approach—combining iterative sprints for technical tasks (e.g., sensor integration, PID tuning) with fixed sub-milestones for team coordination — could have balanced agility and accountability.

Risk Management

Additionally, risks of component failures should have been accounted for during the initial design phase of the buggy. Whether it is by building redundancy into our hardware, such as spare sensor boards, or adding considerations of robustness in the hardware, both would have helped mitigate risks from component failures.

Software Development

From a team organisation perspective, while every member contributed meaningfully throughout the project, differences in technical proficiency,especially in programming, became increasingly apparent during the implementation phase. This disparity limited the degree to which tasks could be parallelised in later stages, creating bottlenecks during system integration.

If given chance to re-do the project, a key aspect is to prioritise early technical up-skilling—particularly in programming and debugging—to support broader participation and reduce dependence on a few individuals.

In addition, establishing a clearer software architecture and investing time in structured planning at an earlier stage would facilitate more effective task delegation. By defining modular components and adopting collaborative strategies such as pair programming, we could improve both code quality and team engagement.

Such planning would also provide clearer guidelines for each team member to follow, improving accountability and making it easier to identify where early investment in skills or effort would yield the most impact.

Conclusion

In conclusion, while our project's planning and organisation successfully delivered key objectives, the challenges of balancing ambition, technical complexity, and time constraints revealed crucial areas for improvement.

Moving forward, we recognise the need to align project scopes with realistic timelines, incorporate redundancy in critical components, and foster a shared technical foundation to ensure balanced team contributions. Clearer sub-goals with defined ownership for each phase would improve accountability and progress tracking, while adopting modular checkpoints—rather than rigid milestones—could better accommodate iterative development. These refinements would create a more adaptive yet structured workflow, better preparing future teams to navigate technical projects efficiently without sacrificing ambition.

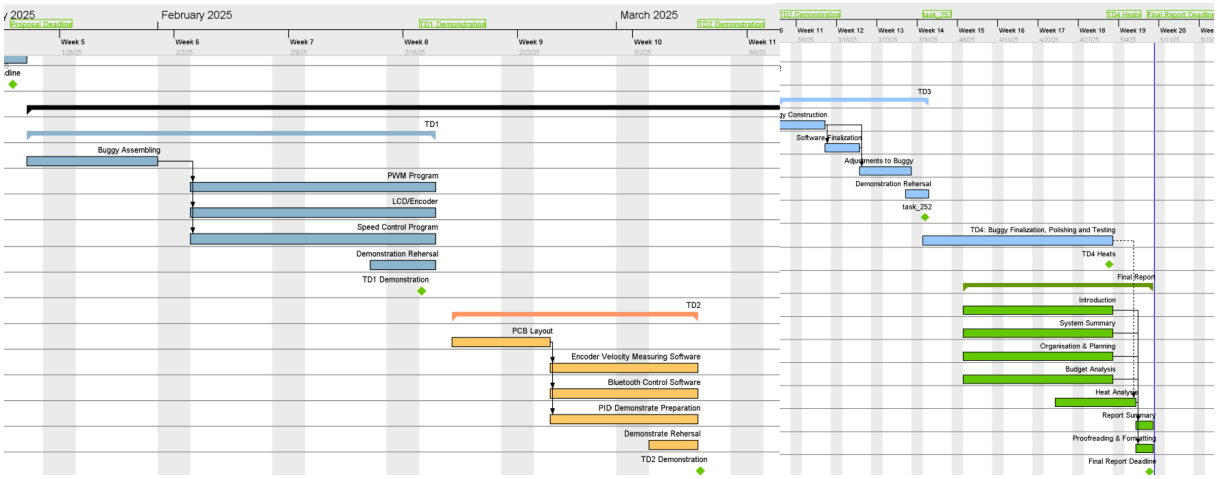

Figure 11: Final Gantt Chart Version

5. Budget vs. Outturn & Business Case

Budget Management

The project operated under a £40 budget, excluding freely provided components such as PCBs, chassis materials, and lab tooling. Total spend was £22.69, focused on performance-critical elements:

Item	Total (£)
3 × 8-Pack Batteries (Duracell)	18.03
20 × Plastic Hex Standoffs (D01495)	2.26
10 × Plastic Screws (97790503111)	2.40
Total Spend	**22.69**
Remaining Budget	**17.31**

Table 1: Breakdown of project expenditures within the £40 budget.

Due to unstable voltage output from rechargeable batteries, non-rechargeable Duracell packs were selected to ensure consistent power during motor tuning. The fresh batteries also offered a competitive edge as they significantly enhanced the power of the motors hence the buggy speed. Plastic hex standoffs and screws were chosen to reduce chassis weight and improve dynamic response. The project remained well under budget while meeting all performance requirements.

Although not counted towards the budget, three iterations of PCB design and a full chassis redesign significantly enhanced the buggy's functionality. These refinements improved circuit reliability, power integrity, and mechanical balance—despite the development inefficiency, they were critical to achieving stable and accurate system performance.

5.1. Market Opportunity

The system targets warehouse automation environments where cost-effective and adaptable AGVs are in growing demand. According to Statzon [4] (2023), the global warehouse automation market is expected to reach $54 billion. Capturing even 1.5% of this market would represent over $820 million in opportunity. The current buggy, while a prototype, demonstrates the potential to meet these demands in small to mid-scale installations, and could also be sold to research institutions or robotics startups as a low-cost AGV development platform with full integration support.

Although the price point exceeds that of typical low-cost line-following buggies, which are available from $45 with 4-sensor configurations [5], this prototype distinguishes itself through its documented API, open-source firmware, and modular development platform. These features provide added technical value and flexibility, making it a

compelling option for users seeking more than basic functionality and aiming to build on top of a robust, well-documented base.

To move towards commercial deployment, further work is required on certification, scaling up manufacturing, and ensuring compatibility with proprietary warehouse systems. These are manageable with additional investment and development, and the current prototype serves as a strong technical foundation for future expansion.

6. Analysis of Heats

6.1. Preparation for the Heats

Despite having a complete code-base and functional app for Bluetooth control and tuning, our buggy was not operational before the Easter break due to a micro-controller failure during TD3, which required full rewiring. Additionally, though we had tuned both motor PID's, the steering PID wasn't fully tuned. As a result of this we performed poorly on TD3 and our buggy would not be functional for the heats.

Over Easter, we reworked both hardware and software. The entire buggy was rewired and cross-checked between our wiring diagram and the board pin-out. The motor drive board, encoders and motors were tested systematically. We wrote code to test the weighted readings of each sensor and found that one of the six sensors was not working, and the buggy would not be able to follow the line. To fix this two sensors had to be removed (to retain symmetry) and the SensorArray class was modified to only use the 4 working sensors, but this in turn created small blind spots between our line sensors that affected the reliability of the line locating system.

The biggest issue faced with hardware was the lack of access to components and tools because the buggy was taken out of campus. In retrospect, making several spare sensors before Easter and ensuring we had adequate spare components (wires, fuses etc.) would've prevented these issues.

Initial tuning efforts used our existing code and the app, but we were not able to run it on mac-OS as we had been using Windows before. As a replacement, values were modified directly in the code instead. Eventually we rewrote the main logic by implementing a state machine and simplifying our control approach by using only one PID rather than 3.

After building a test track, we verified line following behaviour and tuning key values, then the PID values were tuned for a range of base speeds, lastly the Bluetooth turnaround and line-break functionality was implemented.

The buggy was first tuned at a low base speed (30% of maximum values) to ensure that a slow and steady attempt can be made.

The ideal Proportional Gain (k_p) was determined by first setting the Integral Gain (k_i) and Derivative Gain (k_d) to zero. Starting from a very low value, k_p was incrementally increased until the buggy could reliably follow the line, even around sharp corners, with minimal oscillation. This process was then repeated to tune k_i and k_d sequentially.

For each adjustment, we recorded the gain value and observed its effect on the buggy's performance (e.g., responsiveness, overshooting, stability). The tuning process was repeated across multiple base speeds to ensure robustness.

Before the heats we had access to a test track that replicated all of the key parts of the heats track. We began testing the buggy on this track at a low speed but found that the material used for this track had a different reflectivity to the one we had constructed over Easter, so several values are re-tuned. The buggy was working well on the test track and completed the course several times.

6.2. Performance in the Heats

The heats consisted of 2 timed attempts to complete a track. This involved the buggy following the line around a right-hand curve, a left-hand curve, and up a straight inclined section. At the top of the straight inclined section, a single Bluetooth command is sent to execute a $180°$ turn, after which it follows the original path backwards and stops where the line ended.

First Attempt

For attempt 1 we ran the buggy at a moderate speed (50% of its maximum), using the PID values that we had recorded when testing on the mock track the day before at this speed.

The buggy started well, handling both curves and the ramp accurately, but after a successful turn-around the buggy's oscillations caused it to not be centred over the line, and the line was positioned in one of the blind spots caused by the missing sensors. The buggy has completely lost the line, stopping at the slope and thus was disqualified for the attempt.

Having ensured adequate time to replace the sensor array and remodify the code, or having a spare PCB printed before week 10 could have helped to mitigate this.

Second Attempt

A higher base speed (70% of maximum) is considered for the second attempt to maximise the chance of qualifying for the race. However,this is not agreed upon due to an increased risk of losing the line so the Speed and PID Values remained unchanged from the first attempt.

The team also rehearsed the timing of the Bluetooth command, ensuring that it was not issued until the buggy was completely off the ramp. After several practice attempts the turn-around sequence was working smoothly. However the buggy immediately lost the line and veered off course at the first curve due to an increased speed down slope. We were once again disqualified.

A more robust line-following algorithm or the addition of a 'line lost' state that would perform action to find the line again after losing it could have mitigated this. Additionally, more tuning time under heats conditions would have prevented oscillations and overshoot of the line.

Third attempt

The last attempt was untimed. Allowing for partial attempts of the track if the buggy failed. As speed is not of concern,the base speed and PID values are kept as before. The run was much more successful.

The buggy successfully navigated to the top of the ramp, turned, and followed the line down, but failed to handle a curve at the bottom and went off course. Putting the buggy back on the track and restarting it, it completed the rest of the track and came to a timely stop at the end as required. We lost marks for uninterrupted run and the downhill section which could have been prevented by keeping our original 3 PID's for speed control.This would have prevented the buggy from going too fast around the curve at the bottom of the ramp. Another potential strategy was modifying the base speed after the turn-around so that the second half of the run would be done slower.

Despite completing the track in earlier tests, we failed to finish a timed heat and did not qualify for the final race, though we secured most TD4 marks.

6.3. Evaluation of the Buggy's Features

The best features of our buggy were the control algorithm. It was well constructed, including trapezoidal integration, anti-windup protection and low-pass filtering. It was easy to implement the controller, reset its values and tune the gains in the code.

Another successful feature was the chassis design. Most components were well secured, and the sensor board could be secured at several different heights. Additionally it was lightweight, compact and could easily turn around without hitting the sides of the track or getting stuck.

One of the features that could've been improved was the sensor board's robustness. Though it was well designed, having gone through several iterations with major changes, it got damaged easily during excessive testing over Easter. More care should have been taken when soldering the sensors themselves, and perhaps some protective measures could've been incorporated in the chassis design.

Another feature that caused issues was the wiring. As it was changed several times throughout the course of semester 2, it was hard to keep track of what went where. A comprehensive wiring diagram should have been created before the complete assembly of the buggy and before the design document submission.

7. References

[1] Design Report 1, submitted by Group 39, 2024-2025, Available

[2] Design Report 2, Submitted by Group 39, 2024-2025, Available

[3] Proposal Document, Submitted by Group 39, 2024-2025, Available

[4] Statzon,Warehouse Automation Market 2024: Trends, Available

https://statzon.com/insights/the-warehouse-automation-market

[5] Parallax,QTI Line Follower AppKit for the Small Robot, Available https://www.parallax.com/product/qti-line-follower-appkit-for-the-small-robot/?utm_source=chatgpt.com

8. Appendix

8.1. Continuing Professional Development Logs

Jamie Ashton

Current and recent CPD activity:

CPD Activity Title	Description	Dates	CPD Hours
Using Altium Designer	Followed external video series and guides on how to use Altium designer to create a 2-layer PCB	November 2024 - March 2025	14
Circuit prototyping and assembly	Learnt how to prototype a sensor circuit on a breadboard, test it using the NI measurements software suite and assemble it on both strip-board and PCB.	October 2024 - April 2025	20
Learning LTSpice	Followed online tutorials to learn how to use LTSpice and write spice directives to simulate circuits. These included simulating the sensor circuit as well as simulating power electronics circuits such as step-up and step-down convertors.	April 2025 - present	8
Project Euler	A series of programming challenges to test mathematical and algorithm skills in several different programming languages (C,C++, MATLAB)	February 2025 - present	10

Planned and Future CPD activity:

CPD Activity Title	Description	Skills addressed	Dates
3-DOF robotic arm using STM-Nucleo	A planned project to build a 3-DOF robotic arm using STM32 micro-controllers and 9g servos, I plan to implement forward and inverse kinematics to achieve desired poses and to include some Bluetooth control	Embedded systems, C++ programming, kinematics & dynamics	August 2025
RF circuits for SEI Internship	Learning the fundamentals of Rf circuit design for the project 'Commissioning the RHINO blade dipole antenna' with the Jodrell Bank team	Vector Network Analysers, RF circuit design	June - August 2025

Emmanuel Pacholek

Current and recent CPD activity:

CPD Activity Title	Description	Dates	CPD Hours
Programming in C++	I've been making small games such as asteroids, pong, Tetris copycats to increase my proficiency with C++	Since Summer 2024	~400
Programming in Rust	Self-taught language then made things such as a ray-marched renderer (this also required learning about Vulcan and the GPU programming model etc.… as well as different shading languages), HTTP server implementations, desktops apps with Egui front-end to interact with BLE devices, fully destructible 3D world game project in the Bevy Game Engine, recreation of libraries for LCD interaction on the stm32 micro-controller as well as experimentations with embedded rust	Since Summer 2024	~1000
System Verilog	Self-taught System Verilog which is more relevant than VHDL in today's industry and been reading open-source code to learn about the design and architecture of simple CPUs and GPUs	Along 2024/25	~100
Learning Python	Learned the basics of Python as it is a very demanded skill in the electronics industry	Along 2024	~50
Keeping up to date with the industry news	Follow newsletters, YouTube channel and news outlets such as Semi Analysis	2023-now	~300
Volunteering and Networking	Volunteered to help with the National Microelectronics Institute 2025 Conference which gathers all the UK stakeholders of the electronics industry from cutting edge research to suppliers and other product sellers. Was a great opportunity for getting a feel of where the industry is going to in the UK, and I met and talked to plenty of interesting individuals there.	25/03/2025	16
Linux systems	Getting familiar with arch Linux distro as a way to get more proficient and productive with Linux environments which is a useful skill.	2024-now	~150

Planned and Future CPD activity:

CPD Activity Title	Description	Skills addressed	Dates
Industrial Placement Year	I'll be employed for a year at Cadence Designs Systems as an Applications Engineer which is an opportunity to learn about Digital, Analogue and Mixed-Signal Design with ASICs and FPGAs, and get more proficient with software and other skills used in the chip design industry. Great place to meet skilled veterans of the industry and learn from them.	All skills related to chip design engineering and employability I guess	July 2025-July 2026
Programming and CS	Continue getting more proficient via projects especially with Rust and C++, learn about some CS topics such as Data Structures and Algorithms (halfway through), Concurrency and parallel programming, as well as databases and queries	Well programming is almost universally useful, and I enjoy it as a hobby	Always, on the side as a side project
Nano-electronics and Photonics reading/ learning	I'm interested in this stuff and want to read more about it to determine whether or not to do a master in one of these subject areas. I reached out to a researcher of the Photonics department and exchange with them a bit, looking to talk more and maybe see if I can participate to some research when I come back to Uni after my placement.	Knowledge about these subject areas	Now – Graduation in 2027

Junbyung Park

Current and recent CPD activity:

CPD Activity Title	Description	Dates	CPD Hours
Solid Works Simulation	Watched videos on YouTube on how to run the deflection simulations. Tried different simulations to check if the current design is sensible and robust.	Nov, 2024	17 Hours
Deflection Calculation by hand.	Used Young's Modulus and Poisson's ratio. Learnt about deflection curvature equation prior to learning it in semester 2.	Nov, 2024	8 Hours
Learning to use Putty	Tried using putty to connect serial links between the laptop and HM-10 BLE module	Feb, 2025	5 Hours
Soldering	Soldering spare sensors and wires connecting battery pack and a switch	Mar–May, 2025	10 hours

Planned and Future CPD activity:

CPD Activity Title	Description	Skills addressed	Dates
Non-line following Buggy	Demonstrate a buggy driven with Camera and Radar sensors to make my individual buggy.	CAD, Mechanical tools usage, C++	Aug 2025–Dec 2025
Machine Learning Tutorial	Learn coding on how to use machine learning and possibly make a self-driving buggy.	C++, Python, AI	Dec, 2025 – April 2026

Yik Hay Ryan Ho

Current and recent CPD activity:

CPD Activity Title	Description	Dates	CPD Hours
Altium Designer for Initial PCB Design	Went through online tutorials to design the first iteration of our PCB Sensor Array Board.	Winter 2024	5
Soldering	Soldering of Electrical Components	Sep 2024 – May 2025	5
Team-working and Collaboration	Communication within the team, The experience of collaborating extensively for a project with diverse milestones including presentations, reports, laboratory work, and Embedded System Development	Sep 2024 – May 2025	50

Planned and Future CPD activity:

CPD Activity Title	Description	Skills addressed	Dates
Small Unity Projects	Planning to learn and create small 2D games with Unity with a Project-based approach of learning.	C# Programming	Summer 2025
Learning on Wireless Communications	Planning to Research on Communications through online materials and consulting Professors	Programming and Signalling	Summer 2025

Zhe Shang

Current and recent CPD activity:

CPD Activity Title	Description	Dates	CPD Hours
Using the SOLID-WORKS to design a 3D item	Followed videos online to learn how to create basic shapes to the final design.	2024/10 – 2024/11	50+
Buggy assembly	Assembly the buggy that I design and find for improvement.	2025/2 & 2025/3	10
Use the SOLID-WORKS to test the strength	Followed videos online to test the item that I designed	2024/11 & 2025/3	2
Soldering small items	Use soldering to make something like a customised length female to female cable	2025/4 – 2025/5	30+

Planned and Future CPD activity:

CPD Activity Title	Description	Skills addressed	Dates
Design Microelectronic components	I planned to learn to design a simple component by using Silvaco TCAD during my part time job in the holiday.	Use TCAD software	2025/7 – 2025/8
Ultimate Gaming desktop	I planned to design a desk with a PC built in with several feature like wireless charge for phones, a keyboard and a mouse. By using SOLID-WORKS to design, using Geomagic Design X to scan those computer parts like Motherboard GPU etc. And then manufacture it by wood.	Solid-works design, 3D scanning skill, Wood manufacturing skills by the 3D model	2025/6